



Flux de Cost Minim

Flux Maxim de Cost Minim — Lot Seniori

Algolymp

Algolymp Summer School

Problema si Reteaua Reziduala

Edmonds-Karp cu Bellman-Ford

Dijkstra cu Potentiale Johnson

Implementare

Problema Asignarii

Probleme

Problema si Reteaua Reziduala

De ce avem nevoie de flux de cost minim?

Problema transportului

Avem fabrici cu stocuri si depozite cu cereri. Fiecare drum are un **cost per unitate transportata**. Vrem sa satisfacem toate cererile cu **cost total minim**.

Assignment / Matching ponderat

n lucratori, m sarcini, cost c_{ij} pentru $i \rightarrow j$. Gasiti asignarea de cost minim. Caz special direct al FMCM.

Exemple clasice

- Rutare trafic de retea cu costuri de latentă
- Problema cuplajului bipartit ponderat
- Problema k drumuri de cost minim
- Earth Mover's Distance (Wasserstein)
- Scheduling cu deadlines si penalitati

Mesaj cheie

FMCM generalizeaza: cel mai scurt drum, flux maxim, assignment, matching — toate sunt cazuri speciale.

Formulara formală a problemei

Input

Graf orientat $G = (V, E)$, sursa s , destinație t . Fiecare muchie $(u, v) \in E$ are capacitate $\text{cap}(u, v) > 0$ și cost $c(u, v) \in \mathbb{R}$.

Obiectiv: flux maxim de cost minim

Gasiti fluxul $f : E \rightarrow \mathbb{R}_{\geq 0}$ cu $|f|$ **maxim** și **cost total minim**:

$$\min \sum_{(u,v) \in E} c(u, v) \cdot f(u, v)$$

Cu condițiile:

1. **Capacitate:** $0 \leq f(u, v) \leq \text{cap}(u, v)$ pentru orice $(u, v) \in E$
2. **Conservare:** $\sum_w f(u, w) = \sum_w f(w, u)$ pentru orice $u \neq s, t$

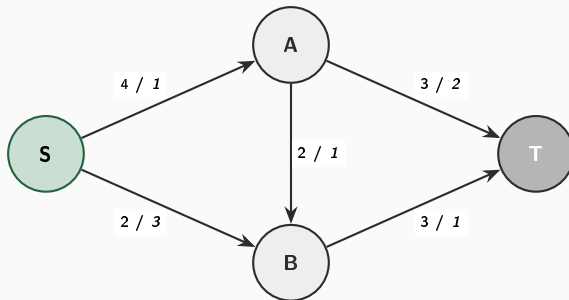
Varianta alternativă: flux de valoare K , cost minim

Gasiti fluxul de valoare **exact** K cu cost minim (nu neaparat maxim). Util cand cerem o cantitate fixa: opriti algoritmul dupa ce $|f| = K$.

Preconditie esentiala

Graful initial G **nu trebuie sa contina cicluri de cost negativ**. Daca exista un ciclu cu cost total < 0 , putem trimite flux arbitrar de mare pe el, scazand costul la $-\infty$ — problema devine nerezolvabila.

Exemplul de lucru



Etichete: **capacitate** / *cost per unitate*

Flux maxim: $|f^*| = 6$. Cum trimitem la **cost**
minim?

Vom urmări acest exemplu pe toată durata lecției.

$S \rightarrow A \rightarrow B \rightarrow T$: cost/unit = 3

$S \rightarrow A \rightarrow T$: cost/unit = 3

$S \rightarrow B \rightarrow T$: cost/unit = 4

Răspuns: cost total = 20

Reteaua reziduala cu costuri

Definitie: reseaua reziduala G_f

Fie f un flux fezabil. Reteaua reziduala $G_f = (V, E_f)$ contine:

- **Muchie directa** (u, v) : capacitate $r(u, v) = \text{cap}(u, v) - f(u, v)$, cost $c(u, v)$ — *daca* $r(u, v) > 0$
- **Muchie inversa** (v, u) : capacitate $r(v, u) = f(u, v)$, cost $-c(u, v)$ — *daca* $f(u, v) > 0$

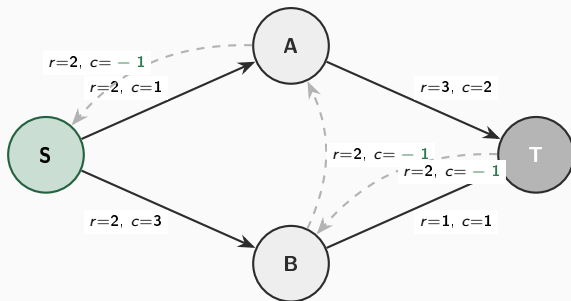
A trimite flux pe (v, u) inseamna a *anula* flux pe (u, v) : economisim $c(u, v)$ per unitate, deci costul muchiei inverse este $-c(u, v)$.

Consecinta importanta: costuri negative!

Spre deosebire de fluxul maxim clasic, reseaua reziduala poate contine muchii cu **cost negativ**. Asta face ca **Dijkstra clasic sa nu fie direct aplicabil**. Avem doua optiuni:

- Bellman-Ford: trateaza costurile negative, dar $O(VE)$ per iteratie
- Dijkstra cu potentiale Johnson: $O(E \log V)$ per iteratie, necesita transformare prealabila

Reteaua reziduala — dupa prima augmentare



Dupa trimiterea a 2 unitati pe $S \rightarrow A \rightarrow B \rightarrow T$: muchia $A \rightarrow B$ e saturata (dispare din reziduala). Apar 3 muchii inverse cu **cost negativ** — ele permit “anularea” (partial) a fluxului trimis anterior.

Edmonds-Karp cu Bellman-Ford

Theorem (Teorema ciclurilor negative)

Un flux fezabil f^ este de cost minim (pentru valoarea sa) **daca si numai daca** reseaua reziduala G_{f^*} nu contine niciun ciclu orientat de cost negativ.*

Demonstratie

- ($\Rightarrow$) Daca G_{f^*} are ciclu negativ C : putem circula $\epsilon > 0$ unitati de flux pe C fara a schimba $|f^*|$. Costul scade cu $\epsilon \cdot |\text{cost}(C)| > 0$. Contradictie cu optimalitatea lui f^* .
- ($\Leftarrow$) Fie f' alt flux cu $|f'| = |f^*|$. Diferenta $f' - f^*$ se descompune in cicluri in G_{f^*} . Daca nu exista cicluri negative: $\text{cost}(f') \geq \text{cost}(f^*)$.

Consecinta directa

Algoritmul trimite flux pe *cel mai scurt drum* (cost minim) din G_f . Dupa augmentare, niciun ciclu negativ nou nu apare in $G_{f'}$ — invariantul de optimalitate se pastreaza. Aceasta e garantia de corectitudine a algoritmului.

Algoritmul Edmonds-Karp cu Bellman-Ford

Pseudocod

1. Initializeaza $f(u, v) = 0$; cost total $C = 0$.
2. **Cat timp** exista drum $s \rightarrow t$ in G_f :
 - 2.1 Gaseste drumul P de cost minim din s in t in G_f prin **Bellman-Ford**.
 - 2.2 $\delta \leftarrow \min_{(u,v) \in P} r(u, v)$ (capacitate maxim trimisibila)
 - 2.3 Trimite δ unitati pe P : actualizeaza fluxul si G_f .
 - 2.4 $C \leftarrow C + \delta \cdot \text{cost}(P)$.
3. Returneaza $(|f|, C)$.

Theorem (Invariantul de optimalitate)

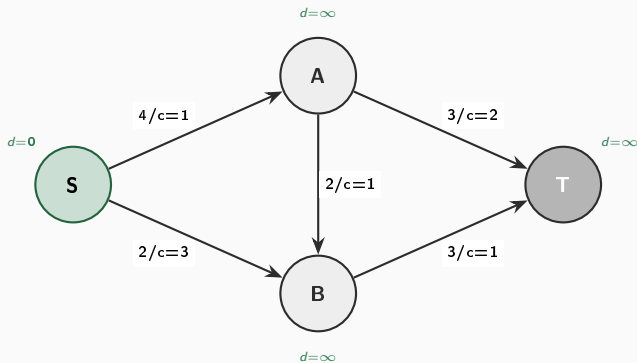
La inceputul fiecarei iteratii, fluxul curent f este de cost minim pentru valoarea sa. Dupa augmentare pe cel mai scurt drum P cu distante $d[\cdot]$, orice ciclu nou care foloseste muchii inverse din P are cost ≥ 0 (prin inegalitatea triunghiului), deci G_f nu capata cicluri negative.

Bellman-Ford — drumul de cost minim

```
1 // d[]=distanțe, pre[]=muchia precedentă
2 d[s] = 0; // restul: infinit
3 for (int iter = 0; iter < n-1; iter++) {
4     bool updated = false;
5     for (int u = 0; u < n; u++) {
6         if (d[u] == INF) continue;
7         for (int idx : g[u]) {
8             auto [v, cap, cost] = e[idx];
9             if (cap <= 0) continue; // saturată
10            if (d[u] + cost < d[v]) {
11                d[v] = d[u] + cost;
12                pre[v] = idx;
13                updated = true;
14            }
15        }
16    }
17    if (!updated) break; // convergența timpurie
18 }
19 // d[t]==INF: niciun drum, flux maxim atins
```

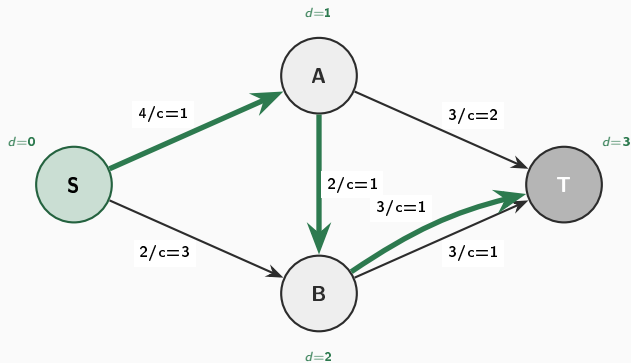
- Funcționează cu costuri negative (spre deosebire de Dijkstra clasic)
- Complexitate $O(V \cdot E)$ per iteratie; în practică converge mult mai repede
- Nu garantează terminarea pe cicluri negative — dar G_f menține invariantul: niciun ciclu negativ!

Exemplul de lucru — Starea initiala



Starea initiala: niciun flux trimis, $f = 0$ pe toate muchiile. Rulam Bellman-Ford din S: $d[A] = 1$, $d[B] = 2$, $d[T] = 3$ (via $S \rightarrow A \rightarrow B \rightarrow T$). Nu exista costuri negative — Bellman-Ford se comporta exact ca Dijkstra.

Iteratia 1 — drumul de cost minim



Drumul ales: $S \rightarrow A \rightarrow B \rightarrow T$

$d[T] = 3$ (via $S \rightarrow A \rightarrow B \rightarrow T$, cost $1 + 1 + 1 = 3$)

Egalitate: $S \rightarrow A \rightarrow T$ are cost $1 + 2 = 3$ de asemenea

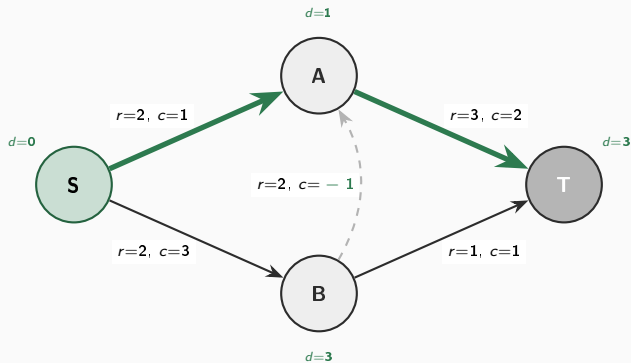
$\delta = \min(4, 2, 3) = 2$ unitati

Augmentare

Cost adaugat: $2 \times 3 = 6$

Flux total: 2 | Cost total: 6

Iteratia 2 — costuri negative in retea reziduala



Bellman-Ford gaseste: $S \rightarrow A \rightarrow T$

Muchia inversa $B \rightarrow A$ cu $c = -1$ e in retea, dar

Bellman-Ford alege drumul optim

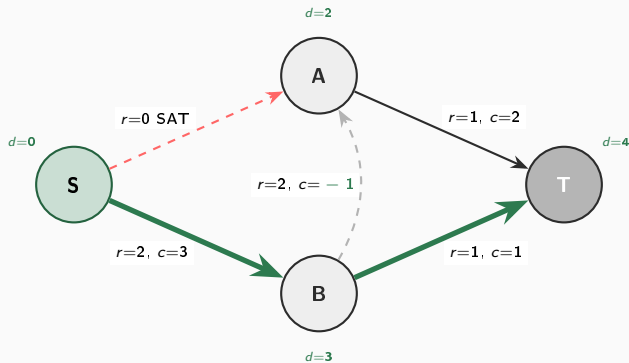
$S \rightarrow A \rightarrow T$: cost $1 + 2 = 3$.

$\delta = \min(2, 3) = 2$

Augmentare

Cost adaugat: $2 \times 3 = 6$

Flux: 4 | Cost: 12



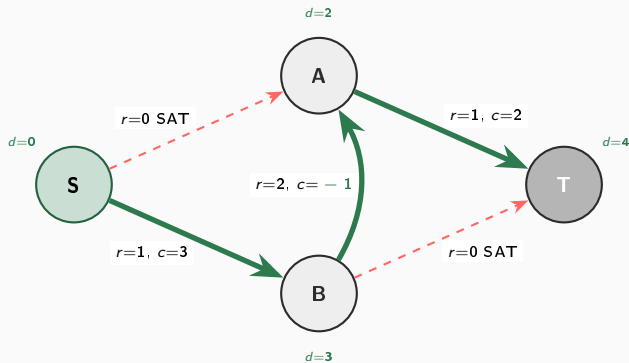
Drum: $S \rightarrow B \rightarrow T$

$S \rightarrow A$ saturat. $d[B] = 3$ (via $S \rightarrow B$).
 $B \rightarrow A$ (cost -1): $d[A] = 2$, $d[T] = 4$ via A.
 $B \rightarrow T$ direct: $d[T] = 4$. Ambele drumuri cost 4.
 Ales: $S \rightarrow B \rightarrow T$. $\delta = \min(2, 1) = 1$

Augmentare

Cost adaugat: $1 \times 4 = 4$
Flux: 5 | Cost: 16

Iteratia 4 — multia inversa in actiune



Drum: $S \rightarrow B \rightarrow A \rightarrow T$ (cu muchie inversa!)

$d[B] = 3$, $d[A] = d[B] + (-1) = 2$, $d[T] = d[A] + 2 = 4$.

Cost real: $3 - 1 + 2 = 4$ /unit. $\delta = \min(1, 2, 1) = 1$.

$B \rightarrow A$ **anuleaza** flux din it. 1 — puterea rețelei reziduale.

Final

Cost adaugat: $1 \times 4 = 4$

Flux: 6 | Cost: 20

Niciun drum $s \rightarrow t$ nu mai exista:

STOP

Rezumatul iteratiilor si complexitatea BF

It.	Drum augmentant	Cost/unit	δ	Cost adaugat
1	$S \rightarrow A \rightarrow B \rightarrow T$	$1 + 1 + 1 = 3$	2	6
2	$S \rightarrow A \rightarrow T$	$1 + 2 = 3$	2	6
3	$S \rightarrow B \rightarrow T$	$3 + 1 = 4$	1	4
4	$S \rightarrow B \rightarrow A \rightarrow T$	$3 + (-1) + 2 = 4$	1	4
Total flux maxim			6	20

Observatii

- Drumurile sunt gasite *in ordine crescatoare de cost*: 3, 3, 4, 4. Proprietate generala a algoritmului.
- Iteratia 4: muchia inversa permite *corectarea* deciziilor anterioare.

Complexitate Edmonds-Karp cu Bellman-Ford

- Bellman-Ford per iteratie: $O(V \cdot E)$ worst-case
- Total: $O(F \cdot V \cdot E)$, $F = |f^*|$
- Periculos pe teste adversariale
- Solutie: **Dijkstra cu potentiale**

Dijkstra cu Potentiale Johnson

De ce avem nevoie de potentiale?

Problema

- Dijkstra e $O(E \log V)$ per iteratie, mult mai bun ca Bellman-Ford ($O(VE)$).
- Dar Dijkstra **nu functioneaza** pe grafuri cu costuri negative!
- Reteaua reziduala are *intotdeauna* costuri negative (muchii inverse).
- Exemplu: muchia $B \rightarrow A$ cu cost -1 din reseaua noastra reziduala.

Solutia: transformarea Johnson

Definim **potentiale** $\pi : V \rightarrow \mathbb{R}$ si **costuri reduse**:

$$\bar{c}(u, v) = c(u, v) + \pi[u] - \pi[v]$$

Alegem π astfel incat obtinem **doua proprietati simultan**:

- $\bar{c}(u, v) \geq 0$ pentru orice muchie — putem aplica Dijkstra!
- **Shortest paths se conserva**: $\text{dist}_{\bar{c}}(s, u) = \text{dist}_c(s, u) + \pi[s] - \pi[u]$
adica diferenta dintre drumuri e constanta $\Rightarrow$ drumul optim ramane acelasi. Nu modificam structura grafului de shortest paths.

Initializarea potentialelor

- Costuri initiale ≥ 0 : $\pi[v] = 0 \ \forall v$. Incepem direct cu Dijkstra.
- Costuri initiale **negative**: rulam Bellman-Ford *o singura data* pe graful initial si setam $\pi[v] = \text{dist}_{BF}(s, v)$. Dupa asta, toate rundele urmatoare folosesc Dijkstra.

Theorem (Costurile reduce sunt non-negative)

Daca $\pi[v] = \text{dist}(s, v)$ (distanțele minime din s în G_f curenta), atunci
 $\bar{c}(u, v) = c(u, v) + \pi[u] - \pi[v] \geq 0$ pentru orice $(u, v) \in G_f$.

Proof.

Prin inegalitatea triunghiului pentru distante minime:

$$\text{dist}(s, v) \leq \text{dist}(s, u) + c(u, v)$$

$$\Leftrightarrow \pi[v] \leq \pi[u] + c(u, v)$$

$$\Leftrightarrow c(u, v) + \pi[u] - \pi[v] \geq 0$$

$$\Leftrightarrow \bar{c}(u, v) \geq 0.$$



Drumurile optime se pastreaza

Costul unui drum P în costuri reduce:

$$\sum_{(u,v) \in P} \bar{c}(u, v) = \sum_{(u,v) \in P} c(u, v) + \pi[s] - \pi[t]$$

Diferența $\pi[s] - \pi[t]$ e constantă pentru orice drum $s \rightarrow t$. Minimizând costuri reduce $\equiv$ minimizând costuri reale.

Actualizarea potentialelor si costul total

Regula de update (dupa fiecare Dijkstra)

Fie $d[v]$ distantele reduse gasite de Dijkstra. Actualizam:

$$\pi'[v] \leftarrow \pi[v] + d[v] \quad \forall v \text{ accesibil din } s$$

Noile potentiale $\pi'[v]$ reprezinta distantele reale de la s la v . Dupa update si augmentare, $\bar{c}(u, v) \geq 0$ pentru orice muchie din $G_{f'}$ (demonstratia foloseste egalitatea $d[v] = d[u] + \bar{c}(u, v)$ pe drumul augmentat).

De ce $\text{cost} += \text{aug} * h[t]$?

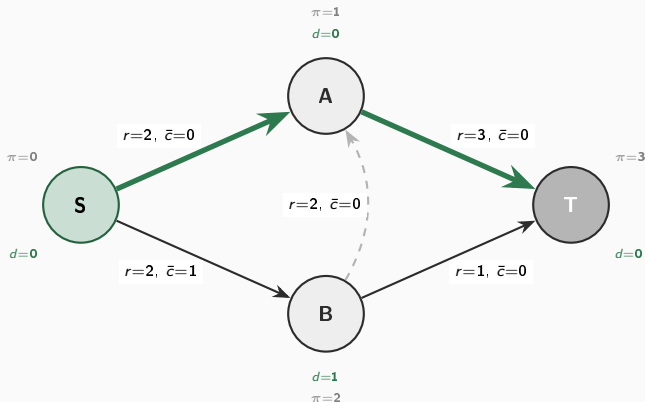
$\pi[s] = 0$ mereu ($d[s] = 0$ la fiecare Dijkstra, $\pi[s]$ ramane 0). Costul real al drumului:

$$\text{cost}_{\text{real}}(P) = d[t] + \pi[t] = \pi'[t]$$

— potentialul lui t dupa update!

Verificare pe exemplu: It. 1: $\pi'[T] = 0 + 3 = 3$, $\text{cost} = 2 \times 3 = 6$. It. 2: $\pi'[T] = 3 + 0 = 3$, $\text{cost} = 2 \times 3 = 6$. It. 3: $\pi'[T] = 3 + 1 = 4$, $\text{cost} = 1 \times 4 = 4$. It. 4: $\pi'[T] = 4 + 0 = 4$, $\text{cost} = 1 \times 4 = 4$. Total: 20. ✓

Dijkstra cu potențiale pe iteratia 2



Costuri reduse $\bar{c}(u, v) = c + \pi[u] - \pi[v]$

$$\bar{c}(S, A) = 1 + 0 - 1 = 0; \bar{c}(A, T) = 2 + 1 - 3 = 0$$

$$\bar{c}(B, A) = -1 + 2 - 1 = 0; \bar{c}(S, B) = 3 + 0 - 2 = 1$$

Dijkstra: toti $\bar{c} \geq 0$, functioneaza corect!

Drum: $S \rightarrow A \rightarrow T$, $\delta = 2$, cost real = $0 + 3 = 3/\text{unit}$

Complexitate finala

$O(E \log V)$ per Dijkstra

F iteratii total

$O(F \cdot E \log V)$

Fata de Bellman-Ford: $O(F \cdot VE)$

Imbunatatire cu factorul $V / \log V$

Dijkstra sau Dinic — dupa potentiale

Dupa potentiale, ambele sunt posibile

- **Dijkstra pe costuri reduse** (varianta standard): la fiecare iteratie gasim un singur drum de cost minim si augmentam. $O(F \cdot E \log V)$ total.
- **Dijkstra cu flux de blocare** (varianta avansata): dupa ce Dijkstra stabileste distantele, augmentam *toate* drumurile de cost minim simultan (exact ca Dinic pe graful de nivel). Reduce numarul de faze Dijkstra.

De ce varianta standard e suficienta pentru competitii

- In competitii, F mic ($\leq 10^4$ – 10^5), $E \leq 10^5$: Dijkstra pe costuri reduse e suficient de rapid.
- Varianta cu flux de blocare e mai complexa de implementat si rareori necesara.
- Implementarea cu vector plat + truc XOR (sectiunea urmatoare) e compacta si rapida.

Tabel comparativ

Edmonds-Karp cu Bellman-Ford	$O(F \cdot VE)$	simplu, lent
Dijkstra + potentiale	$O(F \cdot E \log V)$	recomandat
Dijkstra + flux de blocare	mai rapid	complex, rar

Implementare

Structura si addEdge

```
1 using flow_t = int;
2
3 struct Edge {
4     int from, to;
5     flow_t capacity, flow, cost;
6     Edge(int from, int to, flow_t capacity, flow_t cost)
7         : from(from), to(to), capacity(capacity), flow(0), cost(cost) {}
8 };
9
10 struct MinCostMaxFlow {
11     int V, E = 0;
12     vector<vector<int>>> g;
13     vector<Edge> edges;
14     vector<flow_t> h, dis;
15     vector<int> pre;
16
17     MinCostMaxFlow(int V) : V(V), E(0), g(V) {}
18
19     void addEdge(int from, int to, flow_t capacity, flow_t cost) {
20         g[from].push_back(E++);
21         edges.emplace_back(from, to, capacity, cost);
22         g[to].push_back(E++);
23         edges.emplace_back(to, from, 0, -cost);
24     }
```

Dijkstra cu costuri reduse

```
1  bool dijkstra(int source, int sink) {
2      dis.assign(V, numeric_limits<flow_t>::max());
3      pre.assign(V, -1);
4      priority_queue<pair<flow_t,int>,
5                      vector<pair<flow_t,int>>, greater<>> pq;
6      dis[source] = 0;
7      pq.emplace(0, source);
8      while (!pq.empty()) {
9          auto [d, u] = pq.top(); pq.pop();
10         if (dis[u] != d) continue;
11         for (int i : g[u]) {
12             if (edges[i].flow == edges[i].capacity) continue; // saturated
13             int v = edges[i].to;
14             flow_t w = edges[i].cost + h[u] - h[v]; // reduced cost >= 0
15             if (dis[v] > d + w) {
16                 dis[v] = d + w;
17                 pre[v] = i;
18                 pq.emplace(dis[v], v);
19             }
20         }
21     }
22     return dis[sink] != numeric_limits<flow_t>::max();
23 }
```

Bucla principala — flow()

```
1 // Returns {max_flow, min_cost}
2 pair<flow_t, flow_t> flow(int source, int sink) {
3     h.assign(V, 0);
4     flow_t total_flow = 0, total_cost = 0;
5     while (dijkstra(source, sink)) {
6         for (int i = 0; i < V; i++) h[i] += dis[i]; // update potentials
7         flow_t aug = numeric_limits<flow_t>::max();
8         for (int i = sink; i != source; i = edges[pre[i]^1].to)
9             aug = min(aug, edges[pre[i]].capacity - edges[pre[i]].flow);
10        for (int i = sink; i != source; i = edges[pre[i]^1].to) {
11            edges[pre[i]].flow += aug;
12            edges[pre[i]^1].flow -= aug;
13        }
14        total_flow += aug;
15        total_cost += aug * h[sink]; // h[sink] = real cost of this path
16    }
17    return {total_flow, total_cost};
18 }
19 };
```

XOR pentru perechi de muchii

`addEdge` adauga mereu doua muchii consecutive: directa (i par) si inversa ($i+1$ impar). Pentru a accesa muchia pereche in $O(1)$: `edges[i ^ 1]`.

Reconstructia drumului inapoi

`pre[v]` = indicele muchiei ce *intra* in v . Traversare de la `sink` la `source`:

```
1 for (int i = sink; i != source; i = edges[pre[i]^1].to)
2     // edges[pre[i]] = muchia ce intra in i
3     // edges[pre[i]^1] = muchia inversa (pleaca din i)
4     // edges[pre[i]^1].to = nodul anterior pe drum
```

Costul real dupa actualizarea potentialelor

Dupa `h[i] += dis[i]`, valoarea `h[sink]` reprezinta costul real (in unitati originale) al drumului gasit — deoarece `dis[sink]` e distanta in costuri reduce, iar cumului potentialelor reface costul original.

Flux de valoare exact K

Cand `total_flow + aug` urmează să depășească K , adaugam doar $K - \text{aug}$ flux. Util cand problema cere exact K unitati de flux (nu maximul).

Problema Asignarii

Problema asignarii (Assignment Problem)

Enunt

Avem n lucratori si n joburi. Costul de a asigna lucratorul i la jobul j este $A[i][j]$. Gasiti asignarea bijectiva de cost minim:

$$\min_{\sigma \in S_n} \sum_{i=1}^n A[i][\sigma(i)]$$

Formulari echivalente

- **Matrice $n \times n$:** selectati un element per linie si per coloana, suma minima.
- **Graf bipartit complet $K_{n,n}$** cu ponderi pe muchii: gasiti *perfect matching* de cost minim.
- **Rectangular:** $n \neq m$ — completati cu linii/coloane de cost 0 sau ∞ pentru a aduce la forma patrata.
- **Maximizare:** inmultiti costurile cu -1 .

Aplicatii

Planificare joburi pe masini, alocare resurse, recunoastere de tipare (matching imagini), alinierea secventelor (bioinformatica).

Constructia grafului

- Nod sursa S , nod destinatie T .
- $S \rightarrow \text{lucrator}_i$: capacitate 1, cost 0 (n muchii)
- $\text{Lucrator}_i \rightarrow \text{job}_j$: capacitate 1, cost $A[i][j]$ (n^2 muchii)
- $\text{Job}_j \rightarrow T$: capacitate 1, cost 0 (n muchii)

Solutia

Rulam FMCM pentru n unitati de flux $S \rightarrow T$.

Fiecare unitate de flux alege un drum $S \rightarrow i \rightarrow j \rightarrow T$, adica asigneaza lucratorul i la jobul j .

Complexitate: $O(n^2 \cdot n \log n) = O(n^3 \log n)$ cu Dijkstra + potentiale.

Algoritmul Unguresc (Hungarian Algorithm)

Algoritm specializat pentru asignare in $O(n^3)$, mai rapid decat FMCM in practica. Foloseste potentiale (dual) similar cu Johnson, dar exploateaza structura bipartita. Implementare: cp-algorithms.com/graph/hungarian-algorithm.

FMCM vs. Unguresc: Graf general cu structura de asignare — FMCM. Matrice $n \times n$ pura — Unguresc pentru $O(n^3)$ garantat.

Probleme

Problema

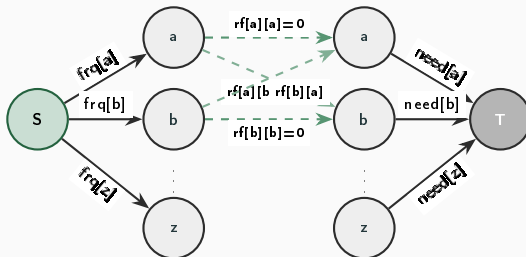
Avem un **stoc** de litere: $freq[c]$ litere de tipul c (pentru $c \in \{a, \dots, z\}$). Vrem sa formam un cuvânt dat W .

Disponem de M **oferte** de la vraci Jaxson: “Da-mi litera L_1 si x galbeni $\rightarrow$ primești litera L_2 ” (schimb 1-la-1).

Minimizati **numarul de galbeni** platit, sau afisati “NU” daca e imposibil. T scenarii de test.

Observatie: Floyd-Warshall pe oferte

Putem înlănțui oferte: $a \xrightarrow{2} b, b \xrightarrow{3} c \Rightarrow$ transformam $a \rightarrow c$ cu cost 5. Precalculam $rf[i][j]$ = costul minim de a transforma litera i in litera j (Floyd-Warshall pe graful ofertelor, 26 noduri). Daca nu exista cale: $rf[i][j] = \infty$.



54 noduri: S, 26 litere stoc, 26 litere nevoie, T

- $S \rightarrow i$: $\text{cap} = \text{freq}[i]$, $\text{cost} = 0$
- $i \rightarrow j$: $\text{cap} = \infty$, $\text{cost} = \text{rf}[i][j]$ (daca $\text{rf}[i][j] < \infty$)
- $j \rightarrow T$: $\text{cap} = \text{need}[j]$, $\text{cost} = 0$

Raspuns

Daca flux maxim = $|W|$: raspuns = cost minim.

Altfel: "NU" (nu avem suficiente litere).

Link

code.algolymp.com/problems/3450

```
1 // Floyd-Warshall pe oferte (26 litere)
2 for (int k = 0; k < 26; k++)
3     for (int i = 0; i < 26; i++)
4         for (int j = 0; j < 26; j++)
5             rf[i][j] = min(rf[i][j], rf[i][k] + rf[k][j]);
6
7 // Constructie FMCM cu 26+26+2 noduri
8 MinCostFlow<ll> mcf(2 * 26 + 2);
9 for (int i = 0; i < 26; i++) {
10     mcf.addEdge(0, i + 1, frq[i], 0);           // S -> stoc[i]
11     for (int j = 0; j < 26; j++)
12         if (rf[i][j] < INF)
13             mcf.addEdge(i+1, j+26+1, INF, rf[i][j]); // stoc[i]->nevoie[j]
14     mcf.addEdge(i + 26 + 1, 53, need[i], 0);    // nevoie[i] -> T
15 }
16 auto [flow, cost] = mcf.flow(0, 53);
17 if (flow != (int)cuvant.size()) cout << "NU\n";
18 else cout << "DA " << cost << "\n";
```

Complexitate

Floyd-Warshall: $O(26^3)$. FMCM: $|W|$ iteratii, $O(26^2)$ muchii, $O(52)$ noduri. Total per test: $O(26^3 + |W| \cdot 26^2 \cdot \log 52)$.

Enunt

Se da un graf orientat cu N noduri si M muchii, fiecare muchie avand o lungime. Gasiti parcursul **inchis de lungime minima** care traverseaza fiecare muchie **cel putin o data**.

Observatie: Circuitul Eulerian

Un graf orientat admite circuit Eulerian daca si numai daca e **tare conex** si **fiecare nod e echilibrat**: $\deg^+(v) = \deg^-(v)$ pentru orice v .

Daca graful e Eulerian, raspunsul e simplu: **suma lungimilor tuturor muchiilor** (un circuit Eulerian le traverseaza pe fiecare exact o data). Problema: graful nu e necesar Eulerian!

Idee: echilibrăm graful cu traversari suplimentare

Adaugam traversari suplimentare (pe traseele cele mai scurte) pentru a echilibra nodurile dezechilibrate. Costul minim al acestor traversari suplimentare = FMCM.

Dezechilibrul unui nod

$\text{dez}(v) = \text{deg}^-(v) - \text{deg}^+(v)$ (in minus out)

- $\text{dez}(v) > 0$: mai multe muchii care intra decat ies, trebuie sa trimita flux (sursa)
- $\text{dez}(v) < 0$: mai multe muchii care ies decat intra, trebuie sa primeasca flux (destinatie)
- $\text{dez}(v) = 0$: echilibrat, nicio actiune necesara

Graf FMCM

1. **Floyd-Warshall**: calculeaza $\text{dist}(u, v) = \text{cel mai scurt drum intre orice pereche}$
2. **Nod sursa S , destinatie T** (noduri auxiliare):
 - $S \rightarrow v$ cu $\text{cap} = \text{dez}(v)$, $\text{cost} = 0$, pentru $\text{dez}(v) > 0$
 - $v \rightarrow T$ cu $\text{cap} = |\text{dez}(v)|$, $\text{cost} = 0$, pentru $\text{dez}(v) < 0$
 - $u \rightarrow v$ cu $\text{cap} = \infty$, $\text{cost} = \text{dist}(u, v)$, daca $\text{dez}(u) > 0$, $\text{dez}(v) < 0$, $\text{dist}(u, v) < \infty$

Raspuns final

$\text{Raspuns} = \sum_{\text{muchii}} \text{lungime} + \text{cost minim FMCM}$

Interpretarea fluxului

- Flux $u \rightarrow v$ = traversare suplimentara pe drumul scurt $u \rightarrow v$
- Adauga $+1$ la $\deg^+(u)$ si $+1$ la $\deg^-(v)$
- Dupa FMCM: graful (original + traversari) devine Eulerian

Detectare imposibilitate

Floyd-Warshall da $\text{dist}(u, v) = \infty$ pt. o pereche necesara $\Rightarrow$ graful nu e conex tare $\Rightarrow$ imposibil.

Complexitate

- Floyd-Warshall: $O(N^3)$
- FMCM: $O(F \cdot N^2 \log N)$, $F \leq M$
- Total: $O(N^3 + M \cdot N^2 \log N)$
- $N \leq 60$, $M \leq 1000$: rapid

Link

infoarena.ro/problema/traseu

Problema

Muzeu cu N camere liniare. Camera i are rezerve infinite de lingouri de valoare v_i si greutate g_i . Intra K hoti, fiecare cu rucsac de capacitate G (initial gol).

La fiecare pas i : toti hotii avanseaza din camera i in camera $i + 1$. Alarma i (intre camerele i si $i + 1$): se declanseaza daca $\geq x_i + 1$ hoti au **aceeasi greutate totala** in rucsac la acel moment.

Maximizati captura totala, sau -1 daca hotii sunt prinsi indiferent ce fac.

Exemplu: $N = 2$, $K = 2$, $G = 3$; **cam. 1:** $v = 10, g = 2, x = 1$; **cam. 2:** $v = 9, g = 1, x = 2$

Hotul 1: ia 1 lingou din cam. 1 (greutate 2) + 1 din cam. 2 (greutate 3, valoare 19).

Hotul 2: ia 3 lingouri din cam. 2 (greutate 3, valoare 27).

Dupa cam. 1: greutati $\{2, 0\}$ — diferite, alarma 1 ($x_1 = 1$) nu suna.

La iesire: greutati $\{3, 3\}$ — alarma 2 cere ≥ 3 cu aceeaasi greutate; doar 2, deci OK.

Total: 46.

Observatia cheie: profilul de greutate

Fiecare hot urmeaza un “profil de greutate”: secventa de greutati $(w_0=0, w_1, \dots, w_N)$ cu $0 \leq w_{i-1} \leq w_i \leq G$ si $(w_i - w_{i-1})$ divizibil cu g_i .

Valoarea unui profil: $\sum_{i=1}^N \frac{w_i - w_{i-1}}{g_i} \cdot v_i$.

Constrangerea alarmei i : cel mult x_i hoti pot avea aceeasi valoare w_i .

Graf de flux: straturi (camere) $\times$ greutati

Noduri (i, w) pentru $0 \leq i \leq N$, $0 \leq w \leq G$.

- $S \rightarrow (0, 0)$: cap = K , cost = 0 (toti hotii incep cu greutate 0)
- $(i-1, w) \rightarrow (i, w')$ cu $w' = w + j \cdot g_i$, $j \geq 0$: cap = K , cost = $-j \cdot v_i$ (negat pentru maximizare), daca $w' \leq G$
- Fiecare (i, w) cu $i \geq 1$: *split* in $(i, w)_{in} \rightarrow (i, w)_{out}$ cu cap = x_i (constrangerea alarmei i ; la $i = 0$ nu exista alarma)
- $(N, w)_{out} \rightarrow T$: cap = K , cost = 0 (iesire cu orice greutate)

Rezolvare

Rulam FMCM pentru K unitati de flux de la S la T .

- Flux maxim $< K$: **output** -1
(capacitatile x_i nu permit K hoti)
- Flux $= K$: raspuns $= -\text{cost_minim}$
(costuri negative = profit maxim)

Exemplu $K = 3$: imposibil

Dupa cam. 1 ($g_1 = 2$, $G = 3$): greutatele posibile 0 sau 2. Cu $x_1 = 1$: max 1 hot/greutate $\Rightarrow$ max 2 hoti. $3 > 2 \Rightarrow \text{flux} = 2 < K = 3 \Rightarrow -1$.

Complexitate

- Noduri: $O(N \cdot G)$
- Muchii: $O(N \cdot G^2)$
- Faze Dijkstra: $\leq K$
- Total: $O(K \cdot N \cdot G^2 \cdot \log(NG))$
- OK pentru $N, G \leq 300$, $K \leq 50$

Link

AlgoCode: Hoata (P. 152)

Problema

Graf bipartit cu $n_1 + n_2$ varfuri si m muchii. Fiecare varf este colorat **R**, **B**, sau **U** (necolorat). Fiecare muchie poate fi colorata **R** (cost r), **B** (cost b), sau lasata **U** (cost 0), astfel incat:

- Varf **R**: $\#muchii_rosii > \#muchii_albastre$ la el
- Varf **B**: $\#muchii_albastre > \#muchii_rosii$ la el
- Varf **U**: nicio constrangere

Gasiti colorarea de cost total minim, sau -1 daca nu exista.

Constrangeri

$n_1, n_2, m, r, b \leq 200$. Graful bipartit: varfuri stanga L si dreapta R .

Idee cheie: colorare = directia fluxului

Fiecare muchie originală (u, v) cu $u \in L$, $v \in R$ devine:

- Muchia colorată **R** = flux de la stanga la dreapta: $u \rightarrow v$
- Muchia colorată **B** = flux de la dreapta la stanga: $v \rightarrow u$
- Muchia **U** = niciun flux pe ea

Punem costuri pe muchii: $u \rightarrow v$ cu $\text{cap} = 1$, $\text{cost} = r$; si $v \rightarrow u$ cu $\text{cap} = 1$, $\text{cost} = b$.

Constrangerile pe varfuri se traduc ca limite inferioare pe flux

Fie $u \in L$ (stanga). Fluxul net OUT din u catre $R = \#rosii - \#albastre$ la u .

- u este **R**: flux net $> 0 \Rightarrow$ adaugam $S \rightarrow u$ cu limita inf. 1, capacitate ∞ , cost 0.
- u este **B**: flux net $< 0 \Rightarrow$ adaugam $u \rightarrow T$ cu limita inf. 1, capacitate ∞ , cost 0.
- u este **U**: adaugam $S \rightarrow u$ si $u \rightarrow T$ ambele cu cap ∞ , cost 0 (echilibram liber).

Varfurile din R (dreapta): simetric (rolurile S si T sunt inversate).

Tehnica fluxului cu cerinte minime

Limita inferioara l pe muchia $x \rightarrow y$: doua muchii:

- $x \rightarrow y$: $\text{cap} = l$, $\text{cost} = -\text{BIG}$ (forteaza saturarea)
- $x \rightarrow y$: $\text{cap} = \infty$, $\text{cost} = 0$ (capacitate optionala)

Alternativ: super-sursa/super-destinatie auxiliara.

Detectare -1

- **Cost negativ**: daca $\text{cost minim} < 0 \Rightarrow$ limite inferioare nesatisfacute $\Rightarrow -1$.
Concret: $\text{cost} < -\text{BIG} \cdot (\#\text{varfuri colorate})$.
- **Super-nod**: daca fluxul maxim pe super-nod auxiliar nu satureaza toate muchiile cu limita inferioara $\Rightarrow -1$.

Complexitate

$V = O(n_1 + n_2)$, $E = O(m + n_1 + n_2)$, $F = O(n_1 + n_2)$. Total: $O(F \cdot E \log V)$.

codeforces.com/contest/1288/problem/F

Lower bounds: USACO Guide — Flow with Lower Bounds

Resurse si Probleme de antrenament

- **cp-algorithms.com** — Flux de Cost Minim — explicatii detaliate, implementari Bellman-Ford si Dijkstra, exemple
- **USACO Guide** — Flux de Cost Minim — perspectiva competitiei, template C++, probleme practice cu solutii
- **Wikipedia** — Minimum-cost flow problem — formulare LP, algoritmi clasici (network simplex, scaling), bibliografie
- **CF Blog 105330** — Tutorial FMCM (by -is-this-fft-) — tratare formala si riguroasa: definitii, conditia de optimalitate, algoritm Bellman-Ford
- **CF Blog 95823** — Potentiale Johnson si FMCM — motivatia potentialelor, derivare, aplicare pentru Dijkstra cu costuri negative
- **cp-algorithms.com** — Bellman-Ford — algoritmul de baza pentru drumuri cu costuri negative
- **USACO Guide** — Flux cu Limite Inferioare — circulatie fezabila, flux fezabil S-T, flux maxim/minim cu lower bounds

- **Traseu** (infoarena) — Chinese Postman pe digraf, reducere la FMCM pe dezechilibre
- **CF 1288F — Red-Blue Graph** — colorare bipartita cu constrangeri, flux cu limite inferioare [editorial]
- **Hoata** (ONI 2022 Baraj 2) — K hoti cu constrangeri de greutate, graful in straturi $\text{greutate} \times \text{camera}$ [editorial]
- **Jamne** (Algolymp Contest F) — transformari de litere, Floyd-Warshall + FMCM pe 54 noduri [solutie] [solutie alternativa]
- **Terenuri 3D** (kilonova) — FMCM pe o structura 3D, modelare netriviala [implementare]
- **CF 818G — Four Melodies** — gaseste 4 melodii disjuncte de lungime maxima intr-un sir de note [editorial]